**MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA**

**Technical University of Moldova Faculty of Computers, Informatics and Microelectronics Department of Software and Automation Engineering**

**Postoronca Dumitru FAF-233**

# Report

Laboratory work n.5

**on Embedded Systems**

*Checked by:*
**A. Martiniuc**, *university assistant*
DISA, FCIM, UTM

Chișinău – 2026

# 1 Purpose of laboratory work

The purpose of this laboratory work is to implement a binary signal acquisition and conditioning system for a digital temperature sensor on the ESP32 microcontroller, using FreeRTOS for concurrent task execution and an LCD display plus STDIO for reporting.

## 1.1 Objectives of the laboratory work

Laboratory work has the following objectives that should be achieved:

- Implementing periodic sensor acquisition from a DS18B20 digital temperature sensor over the OneWire protocol at a configurable recurrence (100 ms), exposing the raw value through an internal `sensor_read()` interface
- Implementing binary signal conditioning with configurable hysteresis between two threshold values (26 °C ON / 24 °C OFF) to prevent alert oscillations near the threshold
- Implementing a debounce filter that requires the threshold condition to be confirmed over a minimum number of consecutive samples ($3 \times 100\,\text{ms} = 300\,\text{ms}$) before the alert state transitions
- Displaying a structured report at a lower recurrence (500 ms) on an LCD 16×2 display and via STDIO, showing the current temperature, threshold values, alert state, and debounce counter
- Structuring the firmware into three independent FreeRTOS tasks (Acquisition, Conditioning, Reporter) communicating through a binary semaphore and a mutex
- Organizing the codebase into three hardware-abstraction layers: MCAL, ECAL, and SRV

## 1.2 Problem definition

The application must be structured in three distinct FreeRTOS tasks:

1. **Task 1 – Sensor Acquisition (TaskAcquisition):** Reads raw temperature data from the DS18B20 sensor via the OneWire protocol every 100 ms using a non-blocking conversion pattern. Stores the result in shared sensor data and gives a binary semaphore to signal the conditioning task.

2. **Task 2 – Signal Conditioning (TaskConditioning):** Blocked on the binary semaphore; wakes immediately when new sensor data arrives. Applies a hysteresis filter (thresholds: ON $\geq 26\,^{\circ}\text{C}$, OFF $\leq 24\,^{\circ}\text{C}$) and a debounce counter (3 consecutive confirmations required) to derive the stable binary alert state. Updates LED indicators: green LED when temperature is within normal range, red LED when the alert is active.

3. **Task 3 – Display and Reporting (TaskReporter):** Runs every 500 ms using `vTaskDelayUntil` for drift-corrected periodicity. Updates the LCD 16×2 display with the current temperature and alert state, and emits a CSV plotter line via `printf`. Every 5 seconds it also prints a full structured report including validity, alert state, debounce counter, and threshold configuration.

# 2 Domain Analysis

## 2.1 Application Context and Utilized Technologies

The primary objective of this laboratory work is to implement a **binary signal acquisition and conditioning system** for a digital temperature sensor using **FreeRTOS** on the ESP32 microcontroller. The application demonstrates a complete sensor pipeline: periodic raw data acquisition, hysteresis-based threshold detection with debounce filtering, and structured reporting on both an LCD display and the serial interface.

The system uses the **DS18B20** one-wire digital temperature sensor. The sensor communicates over the **OneWire** protocol on a single GPIO pin (GPIO 4), and is read through the *DallasTemperature* library in a non-blocking configuration (9-bit resolution, `setWaitForConversion(false)`). This pattern issues the temperature conversion request in one acquisition cycle and retrieves the result in the next, avoiding blocking delays inside the FreeRTOS task.

The signal conditioning stage implements two classical digital signal processing techniques:

- **Hysteresis:** The alert turns *ON* only when the temperature exceeds the upper threshold ($\geq 26\,^{\circ}\mathrm{C}$) and turns *OFF* only when it drops below the lower threshold ($\leq 24\,^{\circ}\mathrm{C}$). While the temperature is between the two thresholds, the alert state remains unchanged. This prevents rapid toggling (chattering) when the signal fluctuates near a single threshold value.

- **Debounce filter:** A transition is only accepted after the desired new state persists for `DEBOUNCE_COUNT` $= 3$ consecutive acquisitions (300 ms). If the condition is not maintained for the full count, the counter resets and no state change occurs.

Task synchronization is achieved through two FreeRTOS primitives:

- A **binary semaphore** (`xSemNewData`) given by TaskAcquisition after each read and taken by TaskConditioning, ensuring the conditioning logic runs exactly once per new sample

- A **mutex** (`xSensorMutex`) protecting all shared sensor data fields against concurrent access by the three tasks

## 2.2 Hardware and Software Components

| Component | Description & Role |
|---|---|
| **ESP32 DevKit** | Dual-core microcontroller running FreeRTOS. Hosts all three application tasks and provides the OneWire-compatible GPIO and I2C peripherals. |
| **DS18B20** | Digital temperature sensor communicating over OneWire protocol on GPIO 4. 9-bit resolution ($\approx 0.5\,^{\circ}\mathrm{C}$), non-blocking conversion ($\approx 94\,\mathrm{ms}$). |
| **Green LED** | Connected to GPIO 25. Lights up when temperature is within the normal range (alert OFF). |
| **Red LED** | Connected to GPIO 26. Lights up when the temperature alert is active (temperature confirmed above 26 °C). |
| **LCD 16×2 I2C** | Connected on the I2C bus (SDA=GPIO 21, SCL=GPIO 22), address 0x27. Displays real-time temperature and alert state at 500 ms intervals. |
| $220\,\Omega$ **Resistors** | Current-limiting resistors for each LED. |
| **Serial-to-USB Interface** | UART at 115200 baud for periodic report output and CSV plotter data. |
| **FreeRTOS** | Built into the ESP32 Arduino core. Provides preemptive task scheduling, binary semaphore, and mutex. |
| **PlatformIO** | Build system and IDE integration used for compilation, upload, and serial monitoring. |
| **Breadboard** | Prototyping base for connecting the DS18B20, LEDs, pull-up resistor, and the ESP32 without soldering. |

## 2.3 System Architecture and Solution Justification

The system is organized into three hardware-abstraction layers following the MCAL / ECAL / SRV pattern:

1. **MCAL (Microcontroller Abstraction Layer):** Contains only `GpioDriver`, which wraps `pinMode`, `digitalWrite`, `digitalRead`, and `analogRead`. All direct register / Arduino hardware calls are isolated here.

2. **ECAL (Electronic Component Abstraction Layer):** Contains component drivers: `TempSensor` (DS18B20 via OneWire), `LedDriver`, `LcdDisplay` (LiquidCrystal_I2C), and `SerialIO`. Each module depends only on MCAL or its own library and exposes a clean C API.

3. **SRV (Service Layer):** Contains `SensorData` (shared data store with mutex/semaphore), and the three FreeRTOS task implementations: `TaskAcquisition`, `TaskConditioning`, and `TaskReporter`.

The data flow proceeds as follows:

1. **TaskAcquisition** reads the DS18B20 sensor every 100 ms, stores the temperature and validity flag in `SensorData` (under mutex), and gives the `xSemNewData` binary semaphore.

2. **TaskConditioning** unblocks immediately on the semaphore, reads the latest temperature, applies the `ProcessChannel` hysteresis + debounce function, updates the alert state in `SensorData`, and drives the green/red LEDs accordingly.

3. **TaskReporter** runs every 500 ms via `vTaskDelayUntil`, reads all shared values under mutex, updates the LCD, emits a CSV line for the serial plotter, and every 5 seconds prints a full structured report to STDIO.

## 2.4 Relevant Case Study

A real-world application of hysteresis-based binary signal conditioning is the **thermostat control system** found in HVAC equipment. A simple single-threshold comparator would cause the heating element to switch on and off repeatedly when the room temperature hovers at exactly the set point, leading to relay wear and unstable temperature. The hysteresis band (e.g., set point $\pm 1\,^{\circ}\mathrm{C}$) ensures the heater stays on until the temperature is clearly above the upper threshold and stays off until clearly below the lower threshold. The debounce timer further prevents brief thermal noise spikes from triggering spurious transitions – a technique directly reflected in the `DEBOUNCE_COUNT` parameter of this laboratory work.

# 3 Design

## 3.1 Architectural sketch

Below is the architectural sketch of the components structured in the diagram that highlights the software and hardware components.

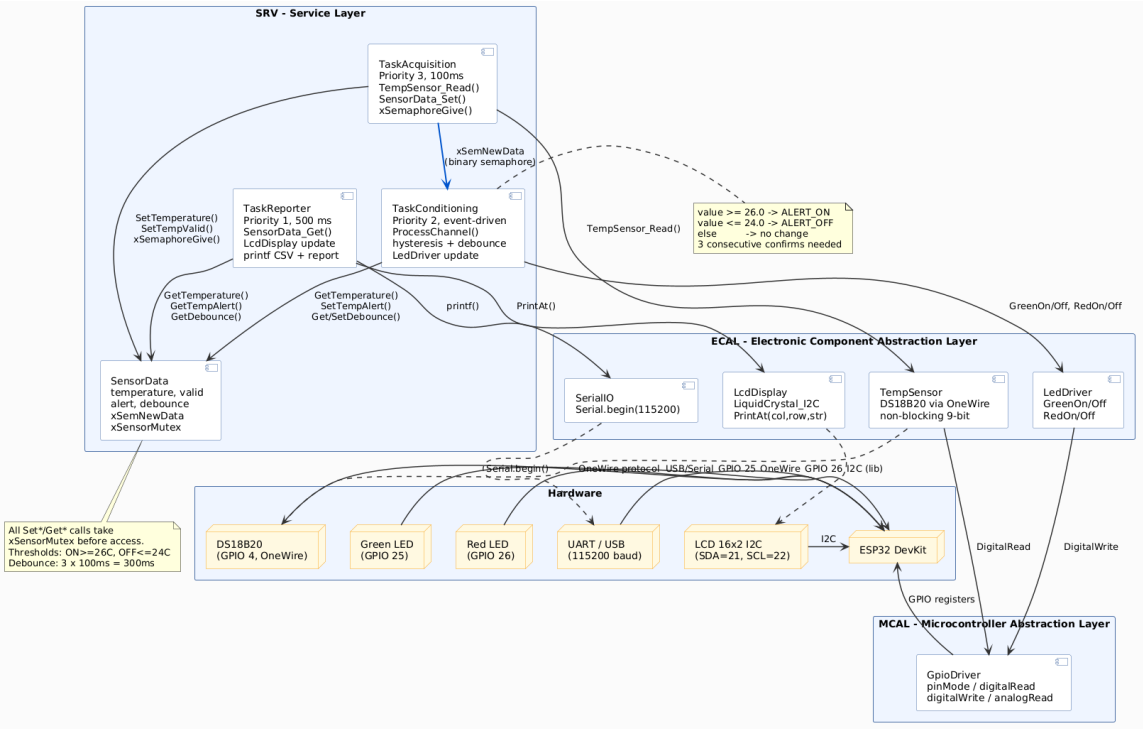


Figure 1: Architectural sketch of the project

This is the description of each of the elements of the diagram:

1. **ESP32 DevKit microcontroller** – central processing unit running FreeRTOS and executing all three application tasks preemptively
2. **DS18B20 temperature sensor (GPIO 4)** – digital sensor communicating over OneWire; read by TaskAcquisition every 100 ms in non-blocking mode
3. **Green LED (GPIO 25)** – output indicator; ON when temperature alert is OFF

4. **Red LED (GPIO 26)** – output indicator; ON when temperature alert is active
5. **LCD 16×2 I2C (SDA=21, SCL=22)** – display updated by TaskReporter every 500 ms with current temperature and alert state
6. **SensorData module** – shared data store protected by `xSensorMutex`; holds temperature value, validity flag, alert state, and debounce counter
7. **TaskAcquisition** – FreeRTOS task (priority 3) that reads the sensor periodically and gives `xSemNewData`
8. **TaskConditioning** – FreeRTOS task (priority 2) blocked on `xSemNewData`; applies hysteresis and debounce, updates LEDs
9. **TaskReporter** – FreeRTOS task (priority 1) that updates LCD and prints STDIO reports at 500 ms intervals
10. **GpioDriver (MCAL)** – wraps all `digitalRead`/`digitalWrite` calls; only layer that touches Arduino hardware APIs directly

## 3.2 Electrical sketch

The circuit diagram outlines the setup for this project, showing the connections between the ESP32, two LEDs (green, red), the DS18B20 temperature sensor, the LCD I2C module, and the computer for serial communication. Each LED is connected to its respective GPIO pin through a 220 Ω current-limiting resistor. The DS18B20 data line (GPIO 4) requires a 4.7 kΩ pull-up resistor to 3.3 V for reliable OneWire communication.

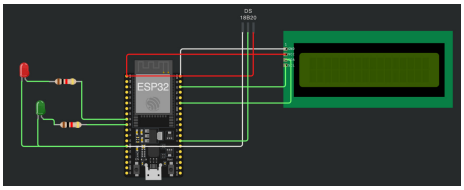


Figure 2: Circuit sketched on wokwi

Figure 2 details the connections and components that make up the system. This visual documentation is crucial for replicating the setup or diagnosing issues, as it clearly shows how each component is integrated into the overall design.

## 3.3 Flow chart

The Figure 3 illustrates the FreeRTOS task interaction flow. TaskAcquisition runs every 100 ms, reads the DS18B20, updates shared data under the mutex, and gives the binary semaphore. TaskConditioning unblocks immediately, reads the shared temperature, applies the `ProcessChannel` hysteresis + debounce logic, and updates the alert state and LEDs. TaskReporter runs every 500 ms independently and reads shared data to refresh the LCD and serial output.

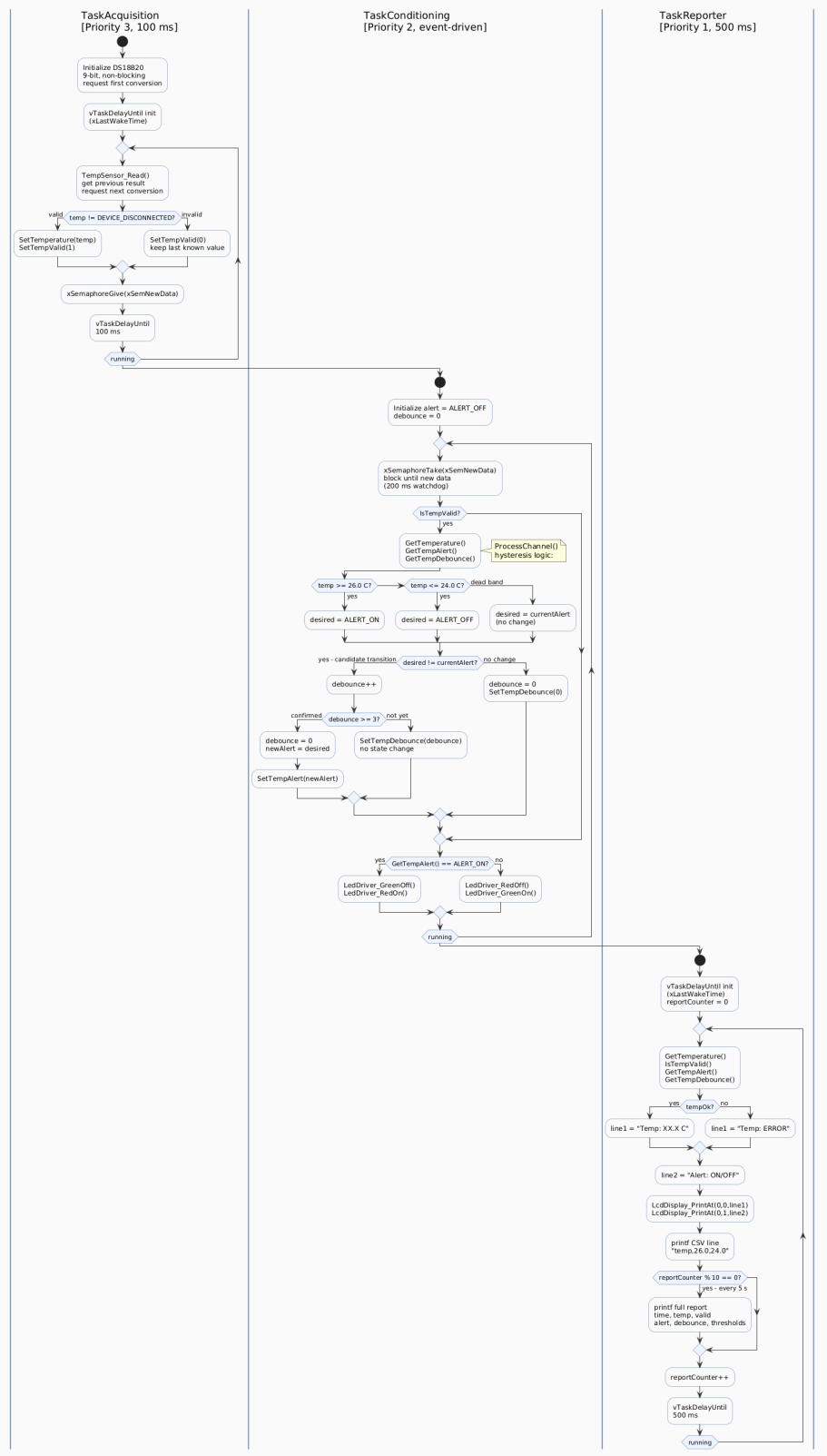


Figure 3: Flow chart of the algorithm

## 3.4 Code design

Code is developed following the principle of separating the interface from the implementation, by creating for each header file `.h` a respective `.cpp` file. The project uses **PlatformIO** with three library directories:

1. **MCAL layer** (`lib/MCAL/GpioDriver`) – hardware-specific code that wraps `Arduino.h` GPIO calls. All direct hardware access is encapsulated here.
2. **ECAL layer** (`lib/ECAL/`) – electronic component drivers: `TempSensor` (DS18B20 via OneWire + DallasTemperature), `LedDriver` (two LEDs), `LcdDisplay` (LiquidCrystal_I2C), and `SerialIO` (UART initialization).
3. **SRV layer** (`lib/SRV/`) – the `SensorData` shared-state module and the three FreeRTOS task implementations.

The `SensorData` module centralizes all shared data behind mutex-protected getters and setters, and owns the FreeRTOS synchronization primitives:

```
// Declared in SensorData.h, initialized in SensorData_Init()
extern SemaphoreHandle_t xSemNewData;   // binary semaphore: new data available
extern SemaphoreHandle_t xSensorMutex; // mutex: protects all shared sensor fields
```

The signal conditioning logic is encapsulated in a single reusable function:

```
static AlertState_t ProcessChannel(
    float value, float threshHigh, float threshLow,
    AlertState_t currentAlert, uint8_t *debounce)
{
    // Hysteresis
    AlertState_t desired;
    if      (value >= threshHigh) desired = ALERT_ON;
    else if (value <= threshLow)  desired = ALERT_OFF;
    else                          desired = currentAlert; // no change in dead band

    // Debounce: require DEBOUNCE_COUNT consecutive confirmations
    if (desired != currentAlert) {
        (*debounce)++;
        if (*debounce >= DEBOUNCE_COUNT) {
            *debounce = 0;
            return desired;  // transition confirmed
```

```
        }
    } else {
        *debounce = 0;        // reset counter if condition does not
   persist
    }
    return currentAlert;      // no state change yet
}
```

TaskReporter uses `vTaskDelayUntil` for drift-corrected 500 ms periodicity and alternates between a compact CSV line (every cycle) and a full structured report (every 5 seconds, i.e. every 10th cycle):

```
// CSV plotter line (every 500 ms)
printf("%.1f,%.1f,%.1f\n", temp, TEMP_THRESH_HIGH, TEMP_THRESH_LOW);

// Full structured report (every 5 s)
if ((reportCounter++ % FULL_REPORT_EVERY) == 0) {
    printf("\n--- SENSOR REPORT (t=%lu ms) ---\n",
           xTaskGetTickCount() * portTICK_PERIOD_MS);
    printf("Temp  : %.1f C (valid=%d, alert=%s, dbg=%d)\n",
           temp, tempOk, (tAlert ? "ON" : "OFF"), tDbg);
    printf("Thresh: ON >= %.1f C, OFF <= %.1f C\n",
           TEMP_THRESH_HIGH, TEMP_THRESH_LOW);
    printf("--------------------------------\n\n");
}
```

# 4 Results

For running the application the following commands were used for compiling, uploading the project and monitoring the port to observe the periodic reports.

```
# list connected boards and their ports
pio device list
# compile and upload to ESP32
pio run --target upload
# open serial monitor at 115200 baud
pio device monitor --baud 115200
```

Below is the output of the commands listed above

```
====================================
  Lab 5 - Temperature Monitor
====================================
Sensor    : DS18B20 (digital) GPIO4
Thresh    : ON >= 26.0C, OFF <= 24.0C
Debounce  : 3 x 100ms = 300ms
Display   : LCD 16x2 I2C + Serial
====================================

23.5,26.0,24.0
23.6,26.0,24.0
23.7,26.0,24.0

--- SENSOR REPORT (t=500 ms) ---
Temp  : 23.5 C (valid=1, alert=OFF, dbg=0)
Thresh: ON >= 26.0 C, OFF <= 24.0 C
------------------------------

26.1,26.0,24.0
26.2,26.0,24.0
26.3,26.0,24.0

--- SENSOR REPORT (t=5500 ms) ---
Temp  : 26.3 C (valid=1, alert=ON, dbg=0)
Thresh: ON >= 26.0 C, OFF <= 24.0 C
------------------------------
```

The system behavior is as follows:

- At startup, the LCD shows `Lab5 TempMonitor` / `Starting...` for 1 second while all modules initialize.
- During normal operation (temperature below 26 °C), the **green LED** is on and the LCD line 1 shows the current temperature (e.g. `Temp: 23.5 C`) while line 2 shows `Alert: OFF`.
- When the temperature rises above 26 °C and the condition is confirmed for 3 consecutive 100 ms samples, the alert transitions to ON: the **red LED** turns on, the green LED turns off, and the LCD line 2 switches to `Alert: ON`.
- The alert turns OFF only after the temperature drops below 24 °C and the lower condition is likewise confirmed for 3 consecutive samples, preventing chattering in the 24–26 °C dead band.
- Every 500 ms, a CSV line is emitted to STDIO in the format `temp,threshHigh,threshLow` for use with a serial plotter.
- Every 5 seconds, a full structured report is printed showing the temperature value, sensor validity, current alert state, debounce counter, and threshold configuration.

Below is the visual representation of the circuit

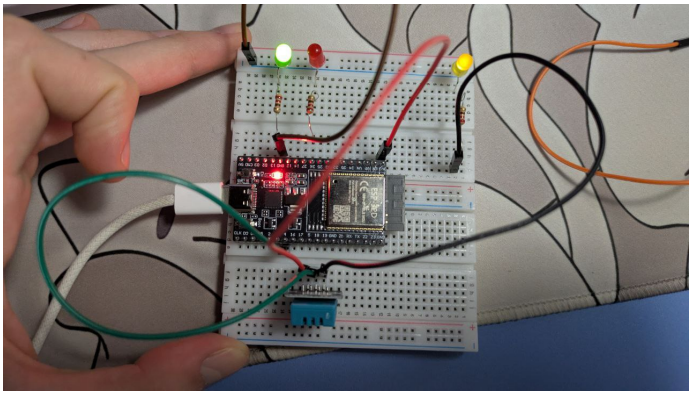

Figure 4: Circuit in idle state

Figure 4 shows the circuit in its idle state with both LEDs off before acquisition begins.

# 5 Conclusion

Completion of this laboratory work has provided practical understanding of binary signal acquisition and conditioning techniques for embedded sensor systems, and reinforced the use of FreeRTOS synchronization primitives for safe inter-task communication.

This fifth laboratory work covered the complete sensor pipeline from raw data acquisition to binary alert generation and structured reporting. The key takeaways are:

1. Implementing **non-blocking sensor acquisition** with the DS18B20 in FreeRTOS context – by using `setWaitForConversion(false)` and requesting the next conversion immediately after reading the previous result, the 94 ms conversion time is hidden inside the 100 ms task period without blocking the task or the scheduler.

2. Applying **hysteresis** to convert a continuous analog signal into a stable binary alert state – the two-threshold design (ON $\geq 26\,^{\circ}\text{C}$, OFF $\leq 24\,^{\circ}\text{C}$) creates a dead band that eliminates chattering when the temperature fluctuates near a single set point.

3. Combining hysteresis with a **debounce counter** – requiring `DEBOUNCE_COUNT` = 3 consecutive confirmations (300 ms) before accepting a state transition prevents brief thermal noise spikes from triggering false alerts, which is an essential requirement in industrial sensor conditioning.

4. Structuring the firmware into three distinct **FreeRTOS tasks** with well-defined roles and priorities: Acquisition (priority 3) runs most frequently and ensures fresh data, Conditioning (priority 2) is event-driven via semaphore and reacts immediately to new samples, and Reporter (priority 1) provides periodic human-readable output without interfering with the control loop.

5. Using a **mutex-protected shared data store** (`SensorData`) as the single source of truth for all sensor state, preventing data races when multiple tasks read or write temperature, alert state, and debounce counters concurrently.

6. Organizing the project into **three hardware-abstraction layers** (MCAL, ECAL, SRV) with PlatformIO, keeping all hardware-specific code in the MCAL layer and making the service layer fully portable across different MCU platforms.

This laboratory work demonstrates how a small set of classical signal processing techniques – hysteresis and debouncing – combined with FreeRTOS synchronization primitives, are sufficient to build a robust and noise-resistant binary event detector from a raw sensor stream.

# References

[1] Espressif Systems. (2023). *ESP32 Series Datasheet*. Retrieved from https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

[2] Maxim Integrated (now Analog Devices). (2019). *DS18B20 Programmable Resolution 1-Wire Digital Thermometer Datasheet*. Retrieved from https://www.analog.com/media/en/technical-documentation/data-sheets/DS18B20.pdf

[3] Stoffregen, P. (2023). *OneWire Arduino Library*. Retrieved from https://github.com/PaulStoffregen/OneWire

[4] Miles Burton. (2023). *Arduino Library for Maxim Temperature ICs (DallasTemperature)*. Retrieved from https://github.com/milesburton/Arduino-Temperature-Control-Library

[5] Real Time Engineers Ltd. (2024). *FreeRTOS API Reference*. Retrieved from https://www.freertos.org/a00106.html

[6] Barry, R. (2016). *Mastering the FreeRTOS Real Time Kernel – A Hands-On Tutorial Guide*. Real Time Engineers Ltd. Retrieved from https://www.freertos.org/Documentation/RTOS_book.html

[7] Barr, M. (2006). *Programming Embedded Systems: With C and GNU Development Tools*. O'Reilly Media, Inc.

# A Source Code

## A.1 Main Entry Point

```
1
2  #include "SerialIO.h"
3  #include "TempSensor.h"
4  #include "LedDriver.h"
5  #include "LcdDisplay.h"
6  #include "SensorData.h"
7  #include "TaskAcquisition.h"
8  #include "TaskConditioning.h"
9  #include "TaskReporter.h"
10
11 void setup()
12 {
13     SerialIoInit();
14     TempSensor_Init();   // DS18B20 on GPIO4, 9-bit non-blocking
15     LedDriver_Init();
16     LcdDisplay_Init();   // LCD 16x2 I2C (0x27, SDA=21, SCL=22)
17     SensorData_Init();   // semaphore + mutex
18
19     printf("\n=====================================\n");
20     printf("  Lab 5 - Temperature Monitor\n");
21     printf("=====================================\n");
22     printf("Sensor    : DS18B20 (digital) GPIO%d\n", TEMP_SENSOR_PIN
       );
23     printf("Thresh    : ON >= %.1fC, OFF <= %.1fC\n",
       TEMP_THRESH_HIGH, TEMP_THRESH_LOW);
24     printf("Debounce  : %d x 100ms = %dms\n", DEBOUNCE_COUNT,
       DEBOUNCE_COUNT * 100);
25     printf("Display   : LCD 16x2 I2C + Serial\n");
26     printf("=====================================\n\n");
27
28     LcdDisplay_PrintAt(0, 0, "Lab5 TempMonitor");
29     LcdDisplay_PrintAt(0, 1, "Starting...");
30     vTaskDelay(pdMS_TO_TICKS(1000));
31
32     // Task priorities: Acquisition=3 (highest), Conditioning=2,
       Reporter=1
33     xTaskCreate(TaskAcquisition_Task,  "Acquisition",  2048, NULL,
       3, NULL);
34     xTaskCreate(TaskConditioning_Task, "Conditioning", 2048, NULL,
       2, NULL);
```

```
35      xTaskCreate(TaskReporter_Task,      "Reporter",      4096, NULL,
        1, NULL);
36  }
37
38  void loop()
39  {
40      vTaskDelay(portMAX_DELAY);
41  }
```

Listing 1: main.cpp - Application Entry Point

## A.2 MCAL Layer

```
1  #pragma once
2  #include <Arduino.h>
3
4  void    GpioDriver_PinMode(uint8_t pin, uint8_t mode);
5  void    GpioDriver_Write(uint8_t pin, uint8_t val);
6  uint8_t GpioDriver_Read(uint8_t pin);
7  void    GpioDriver_Toggle(uint8_t pin);
```

Listing 2: GpioDriver.h - GPIO Abstraction Header

```
1  #include "GpioDriver.h"
2
3  void GpioDriver_PinMode(uint8_t pin, uint8_t mode)
4  {
5      pinMode(pin, mode);
6  }
7
8  void GpioDriver_Write(uint8_t pin, uint8_t val)
9  {
10     digitalWrite(pin, val);
11 }
12
13 uint8_t GpioDriver_Read(uint8_t pin)
14 {
15     return digitalRead(pin);
16 }
17
18 void GpioDriver_Toggle(uint8_t pin)
19 {
20     digitalWrite(pin, !digitalRead(pin));
21 }
```

Listing 3: GpioDriver.cpp - GPIO Abstraction Implementation

## A.3 ECAL Layer – Sensors and Peripherals

```
#pragma once

#define TEMP_SENSOR_PIN 4

void  TempSensor_Init(void);
float TempSensor_Read(void);
int   TempSensor_IsValid(void);
```

Listing 4: TempSensor.h - DS18B20 Driver Header

```
#include "TempSensor.h"
#include <OneWire.h>
#include <DallasTemperature.h>

static OneWire           oneWire(TEMP_SENSOR_PIN);
static DallasTemperature sensors(&oneWire);
static float             lastTemp = 20.0f;
static int               lastValid = 0;

void TempSensor_Init(void)
{
    sensors.begin();
    sensors.setResolution(9);              // 9-bit: ~94ms conversion
    sensors.setWaitForConversion(false);   // non-blocking
    sensors.requestTemperatures();         // start first conversion
    delay(100);
}

float TempSensor_Read(void)
{
    // Non-blocking: get previous result, request next
    float temp = sensors.getTempCByIndex(0);
    lastValid = (temp != DEVICE_DISCONNECTED_C) ? 1 : 0;
    if (lastValid) {
        lastTemp = temp;
    }
    sensors.requestTemperatures();  // queue next read
    return lastTemp;
}
```

```
30
31 int TempSensor_IsValid(void)
32 {
33     return lastValid;
34 }
```

Listing 5: TempSensor.cpp - DS18B20 Driver Implementation

```
1 #pragma once
2
3 void LedDriver_Init(void);
4 void LedDriver_GreenOn(void);
5 void LedDriver_GreenOff(void);
6 void LedDriver_RedOn(void);
7 void LedDriver_RedOff(void);
```

Listing 6: LedDriver.h - LED Driver Header

```
1  #include "LedDriver.h"
2  #include "GpioDriver.h"
3
4  #define LED_GREEN_PIN  25
5  #define LED_RED_PIN    26
6
7  void LedDriver_Init(void)
8  {
9      GpioDriver_PinMode(LED_GREEN_PIN, OUTPUT);
10     GpioDriver_PinMode(LED_RED_PIN,   OUTPUT);
11     GpioDriver_Write(LED_GREEN_PIN, LOW);
12     GpioDriver_Write(LED_RED_PIN,   LOW);
13 }
14
15 void LedDriver_GreenOn(void)  { GpioDriver_Write(LED_GREEN_PIN, HIGH
       ); }
16 void LedDriver_GreenOff(void) { GpioDriver_Write(LED_GREEN_PIN, LOW)
       ;  }
17 void LedDriver_RedOn(void)    { GpioDriver_Write(LED_RED_PIN,   HIGH
       ); }
18 void LedDriver_RedOff(void)   { GpioDriver_Write(LED_RED_PIN,   LOW)
       ;  }
```

Listing 7: LedDriver.cpp - LED Driver Implementation

```
1 #pragma once
2 #include <stdint.h>
3
```

```
void LcdDisplay_Init(void);
void LcdDisplay_Clear(void);
void LcdDisplay_SetCursor(uint8_t col, uint8_t row);
void LcdDisplay_Print(const char *str);
void LcdDisplay_PrintAt(uint8_t col, uint8_t row, const char *str);
```

Listing 8: LcdDisplay.h - LCD I2C Driver Header

```
#include "LcdDisplay.h"
#include <LiquidCrystal_I2C.h>

// I2C address 0x27 (or 0x3F); cols=16, rows=2
// For ESP32: SDA=21, SCL=22
static LiquidCrystal_I2C lcd(0x27, 16, 2);

void LcdDisplay_Init(void)
{
    lcd.init();
    lcd.backlight();
    lcd.clear();
}

void LcdDisplay_Clear(void)
{
    lcd.clear();
}

void LcdDisplay_SetCursor(uint8_t col, uint8_t row)
{
    lcd.setCursor(col, row);
}

void LcdDisplay_Print(const char *str)
{
    lcd.print(str);
}

void LcdDisplay_PrintAt(uint8_t col, uint8_t row, const char *str)
{
    lcd.setCursor(col, row);
    lcd.print(str);
}
```

Listing 9: LcdDisplay.cpp - LCD I2C Driver Implementation

```
#pragma once
```

```
2
3 #define SERIAL_BAUD 115200
4
5 void SerialIoInit(void);
```

Listing 10: SerialIO.h - Serial Initialization Header

```
1 #include "SerialIO.h"
2 #include <Arduino.h>
3
4 void SerialIoInit(void)
5 {
6     Serial.begin(SERIAL_BAUD);
7     delay(2000);
8 }
```

Listing 11: SerialIO.cpp - Serial Initialization Implementation

## A.4 SRV Layer – Shared Data

```
1  #pragma once
2  #include <stdint.h>
3  #include <freertos/FreeRTOS.h>
4  #include <freertos/semphr.h>
5
6  // Thresholds (hysteresis)
7  #define TEMP_THRESH_HIGH   26.0f
8  #define TEMP_THRESH_LOW    24.0f
9
10 #define DEBOUNCE_COUNT     3   // 3 x 100ms = 300ms debounce
11
12 // Alert state
13 typedef enum {
14     ALERT_OFF = 0,
15     ALERT_ON  = 1
16 } AlertState_t;
17
18 // Synchronization primitives
19 extern SemaphoreHandle_t xSemNewData;
20 extern SemaphoreHandle_t xSensorMutex;
21
22 void  SensorData_Init(void);
23
24 // Temperature
25 void         SensorData_SetTemperature(float val);
```

```
float         SensorData_GetTemperature(void);
void          SensorData_SetTempValid(int valid);
int           SensorData_IsTempValid(void);
void          SensorData_SetTempAlert(AlertState_t alert);
AlertState_t  SensorData_GetTempAlert(void);
void          SensorData_SetTempDebounce(uint8_t val);
uint8_t       SensorData_GetTempDebounce(void);
```

Listing 12: SensorData.h - Shared Sensor Data Header

```
#include "SensorData.h"

SemaphoreHandle_t xSemNewData  = NULL;
SemaphoreHandle_t xSensorMutex = NULL;

static float        gTemperature  = 20.0f;
static int          gTempValid    = 0;
static AlertState_t gTempAlert    = ALERT_OFF;
static uint8_t      gTempDebounce = 0;

void SensorData_Init(void)
{
    xSemNewData  = xSemaphoreCreateBinary();
    xSensorMutex = xSemaphoreCreateMutex();
}

void SensorData_SetTemperature(float val)
{
    xSemaphoreTake(xSensorMutex, portMAX_DELAY);
    gTemperature = val;
    xSemaphoreGive(xSensorMutex);
}

float SensorData_GetTemperature(void)
{
    xSemaphoreTake(xSensorMutex, portMAX_DELAY);
    float val = gTemperature;
    xSemaphoreGive(xSensorMutex);
    return val;
}

void SensorData_SetTempValid(int valid)
{
    xSemaphoreTake(xSensorMutex, portMAX_DELAY);
    gTempValid = valid;
    xSemaphoreGive(xSensorMutex);
```

```
}

int SensorData_IsTempValid(void)
{
    xSemaphoreTake(xSensorMutex, portMAX_DELAY);
    int val = gTempValid;
    xSemaphoreGive(xSensorMutex);
    return val;
}

void SensorData_SetTempAlert(AlertState_t alert)
{
    xSemaphoreTake(xSensorMutex, portMAX_DELAY);
    gTempAlert = alert;
    xSemaphoreGive(xSensorMutex);
}

AlertState_t SensorData_GetTempAlert(void)
{
    xSemaphoreTake(xSensorMutex, portMAX_DELAY);
    AlertState_t val = gTempAlert;
    xSemaphoreGive(xSensorMutex);
    return val;
}

void SensorData_SetTempDebounce(uint8_t val)
{
    xSemaphoreTake(xSensorMutex, portMAX_DELAY);
    gTempDebounce = val;
    xSemaphoreGive(xSensorMutex);
}

uint8_t SensorData_GetTempDebounce(void)
{
    xSemaphoreTake(xSensorMutex, portMAX_DELAY);
    uint8_t val = gTempDebounce;
    xSemaphoreGive(xSensorMutex);
    return val;
}
```

Listing 13: SensorData.cpp - Shared Sensor Data Implementation

## A.5 SRV Layer – FreeRTOS Tasks

```
#pragma once
```

```
#include <freertos/FreeRTOS.h>
#include <freertos/task.h>

void TaskAcquisition_Task(void *pvParameters);
```

Listing 14: TaskAcquisition.h - Acquisition Task Header

```
#include "TaskAcquisition.h"
#include "TempSensor.h"
#include "SensorData.h"

#define ACQUISITION_PERIOD_MS 100  // Matches DS18B20 ~94ms conversion

void TaskAcquisition_Task(void *pvParameters)
{
    TickType_t xLastWakeTime = xTaskGetTickCount();

    for (;;) {
        // Read temperature from DS18B20
        float temp = TempSensor_Read();
        SensorData_SetTemperature(temp);
        SensorData_SetTempValid(TempSensor_IsValid());

        // Signal conditioning task that new data is available
        xSemaphoreGive(xSemNewData);

        vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(ACQUISITION_PERIOD_MS));
    }
}
```

Listing 15: TaskAcquisition.cpp - Acquisition Task Implementation

```
#pragma once
#include <freertos/FreeRTOS.h>
#include <freertos/task.h>

void TaskConditioning_Task(void *pvParameters);
```

Listing 16: TaskConditioning.h - Conditioning Task Header

```
#include "TaskConditioning.h"
#include "SensorData.h"
#include "LedDriver.h"

```

```
// Apply hysteresis + debounce for the temperature channel.
static AlertState_t ProcessChannel(
    float value, float threshHigh, float threshLow,
    AlertState_t currentAlert, uint8_t *debounce)
{
    // Hysteresis
    AlertState_t desired;
    if      (value >= threshHigh) desired = ALERT_ON;
    else if (value <= threshLow)  desired = ALERT_OFF;
    else                          desired = currentAlert;

    // Debounce
    if (desired != currentAlert) {
        (*debounce)++;
        if (*debounce >= DEBOUNCE_COUNT) {
            *debounce = 0;
            return desired;  // transition confirmed
        }
    } else {
        *debounce = 0;
    }
    return currentAlert;  // no change yet
}

void TaskConditioning_Task(void *pvParameters)
{
    for (;;) {
        // Block until Acquisition signals new data (200ms watchdog
   timeout)
        if (xSemaphoreTake(xSemNewData, pdMS_TO_TICKS(200)) !=
   pdTRUE) {
            continue;
        }

        // Process temperature channel
        if (SensorData_IsTempValid()) {
            float        temp   = SensorData_GetTemperature();
            AlertState_t tAlert = SensorData_GetTempAlert();
            uint8_t      tDbg   = SensorData_GetTempDebounce();

            AlertState_t newAlert = ProcessChannel(
                temp, TEMP_THRESH_HIGH, TEMP_THRESH_LOW, tAlert, &
   tDbg);

            SensorData_SetTempDebounce(tDbg);
            if (newAlert != tAlert) {
```

```
                SensorData_SetTempAlert(newAlert);
            }
        }

        // Apply LEDs: Green = OK, Red = temperature alert
        AlertState_t ta = SensorData_GetTempAlert();
        if (ta == ALERT_ON) {
            LedDriver_GreenOff();
            LedDriver_RedOn();
        } else {
            LedDriver_RedOff();
            LedDriver_GreenOn();
        }
    }
}
```

Listing 17: TaskConditioning.cpp - Conditioning Task Implementation

```
#pragma once
#include <freertos/FreeRTOS.h>
#include <freertos/task.h>

void TaskReporter_Task(void *pvParameters);
```

Listing 18: TaskReporter.h - Reporter Task Header

```
#include "TaskReporter.h"
#include "SensorData.h"
#include "LcdDisplay.h"
#include <stdio.h>

#define REPORTER_PERIOD_MS  500
#define FULL_REPORT_EVERY   10  // 10 x 500ms = 5 sec

void TaskReporter_Task(void *pvParameters)
{
    TickType_t xLastWakeTime = xTaskGetTickCount();
    uint32_t reportCounter = 0;

    for (;;) {
        float        temp   = SensorData_GetTemperature();
        int          tempOk = SensorData_IsTempValid();
        AlertState_t tAlert = SensorData_GetTempAlert();
        uint8_t      tDbg   = SensorData_GetTempDebounce();

        // ===== LCD =====
```

```
21          char line1[17], line2[17];
22          if (tempOk) {
23              snprintf(line1, sizeof(line1), "Temp: %.1f C    ", temp)
    ;
24          } else {
25              snprintf(line1, sizeof(line1), "Temp: ERROR     ");
26          }
27          snprintf(line2, sizeof(line2), "Alert: %-8s",
28                   (tAlert == ALERT_ON) ? "ON" : "OFF");
29
30          LcdDisplay_PrintAt(0, 0, line1);
31          LcdDisplay_PrintAt(0, 1, line2);
32
33          // ===== Serial plotter (CSV) =====
34          // Format: temp,threshHigh,threshLow
35          printf("%.1f,%.1f,%.1f\n", temp, TEMP_THRESH_HIGH,
    TEMP_THRESH_LOW);
36
37          // ===== Structured report every 5 seconds =====
38          if ((reportCounter++ % FULL_REPORT_EVERY) == 0) {
39              printf("\n--- SENSOR REPORT (t=%lu ms) ---\n",
40                     xTaskGetTickCount() * portTICK_PERIOD_MS);
41              printf("Temp  : %.1f C (valid=%d, alert=%s, dbg=%d)\n",
42                     temp, tempOk, (tAlert ? "ON" : "OFF"), tDbg);
43              printf("Thresh: ON >= %.1f C, OFF <= %.1f C\n",
44                     TEMP_THRESH_HIGH, TEMP_THRESH_LOW);
45              printf("--------------------------------\n\n");
46          }
47
48          vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(
    REPORTER_PERIOD_MS));
49      }
50  }
```

Listing 19: TaskReporter.cpp - Reporter Task Implementation